Digital Logic & Computer Organization

Topic 01: Memory Hierarchy Design & Performance

Department of Computer Science & Engineering

Faculty: Prof. Computer Architecture

Interactive Seminar Presentation Deck

Overview:

- Need for Memory Hierarchy & The Memory Wall
- The Multi-Tier Pyramid Structure
- Locality of Reference (Temporal & Spatial)
- Effective Memory Access Time (EMAT) & Numerical Analysis

1. The Need for Memory Hierarchy

Bridging the Processor-Memory Gap

Key Challenges in Modern Systems:

- Speed Divergence: CPU frequency scales faster than DRAM latency.
- Cost Constraint: Fast memory (SRAM) is costly; large memory (DRAM) is slower.
- Capacity Demands: Modern workloads require gigabytes of address space.
- The Ideal Memory: As fast as SRAM, as large as disk, as cheap as flash.
- Solution: Layered hierarchy leveraging caching and spatial/temporal locality.

2. The Memory Hierarchy Pyramid

Levels, Access Times & Typical Capacities

Level 0 - CPU Internal Registers : 0.25 - 0.5 ns | ~1 KB | Compiler Managed

Level 1 - L1 Cache (I-Cache/D-Cache) : 0.5 - 1.5 ns | 32-128 KB | Hardware Managed

Level 2 - L2 Cache (Per Core) : 3.0 - 7.0 ns | 256KB-1MB | Hardware Managed

Level 3 - L3 Shared Cache : 10 - 25 ns | 8 - 64 MB | Hardware Managed

Level 4 - Main Memory (DRAM) : 50 - 80 ns | 8 - 64 GB | OS & Hardware Managed

Level 5 - Solid-State Drive (SSD) : 10 - 50 us | 512GB - 2TB | File System Managed

Level 6 - Cloud & Tertiary Archival : >100 ms | Unlimited | Distributed Storage

3. Locality of Reference Principle

The Foundation of Caching Effectiveness

Temporal Locality (Locality in Time):

- If a memory location is accessed, it will be accessed again soon.
- Examples: Loop counters, recurrent variables, subroutines.

Spatial Locality (Locality in Space):

- If a location is referenced, adjacent locations are likely to be accessed.
- Examples: Sequential instruction fetch, array iterations, struct records.

Empirical Law: 90/10 Rule - 90% of execution time is spent in 10% of the code.

4. Effective Memory Access Time (EMAT)

Formula & Numerical Analysis

Basic Two-Tier Formula:

EMAT = h * Tc + (1 - h) * (Tc + Tm)

Where: h = Hit Ratio, Tc = Cache Access Time, Tm = Main Memory Time

Numerical Demonstration:

Given Tc = 2 ns, Tm = 50 ns

If h = 0.90 (90% hit rate):

EMAT = 0.90 * 2 + 0.10 * 52 = 1.8 + 5.2 = 7.0 ns

If h = 0.98 (98% hit rate):

EMAT = 0.98 * 2 + 0.02 * 52 = 1.96 + 1.04 = 3.0 ns

Conclusion: An 8% hit rate increase more than doubles performance!

5. Summary and Review Questions

Key Takeaways for Students

Takeaway 1: Memory hierarchy hides latency through predictive caching.

Takeaway 2: Lower tiers provide non-volatile, economical bulk storage.

Takeaway 3: Cache replacement and mapping determine real-world throughput.

Seminar Review Questions:

1. Differentiate between spatial and temporal locality with C++ examples.
2. Compute EMAT for a 3-tier system (L1, L2, DRAM).
3. Explain why registers cannot replace cache memories entirely.